

Practical 3

AIM: To Implementation of max-heap sort algorithm

1 ALGORITHM:

Max Heap Algorithm

1. Start.

2. Read n elements into an array A .

3. Set $i = \lfloor n/2 \rfloor$ (last non-leaf node).

4. Compare the node $A[i]$ with its left and right children.

5. Find the largest element among the parent, left child, and right child.

6. If the largest element is not the parent, swap the parent with the largest child.

7. Apply MAX-HEAPIFY to the affected child position.

8. Decrease i by 1.

9. Repeat steps 4-8 until $i = 0$.

10. The array now satisfies the Max Heap property.

11. Stop.

2 PROGRAM:

```
#include <iostream>
```

```
using namespace std;
```

```
void heapify(int arr[], int n, int i)
```

```
{
```

```
    int largest = i;
```

```
    int left = 2 * i + 1;
```

```
    int right = 2 * i + 2;
```

```
if (left < n && arr[left] > arr[largest])
largest = left;
if (right < n && arr[right] > arr[largest])
largest = right;
if (largest != i){
int temp = arr[i];
arr[i] = arr[largest];
arr[largest] = temp;
heapify(arr, n, largest);
}}
void heapSort(int arr[], int n){
// Build Max Heap
for (int i= n/2-1;i >= 0;i--){
heapify(arr, n, i);
// Heap Sort
for (int i=n-1;i>0;i--){
int temp = arr[0];
arr[0] = arr[i];
arr[i] = temp;
heapify(arr, i, 0);}}
int main(){
int arr[] = {12, 11, 13, 5, 6, 7};
int n = 6;
cout << "Original array: ";
for (int i = 0; i < n; i++)
cout << arr[i] << " ";
heapSort(arr, n);
cout << "\nSorted array: ";
for (int i = 0; i < n; i++)
cout << arr[i] << " ";
return 0;}
```

3 OUTPUT:

```
C:\WINDOWS\system32\cmd. × + v
Original array: 12 11 13 5 6 7
Sorted array: 5 6 7 11 12 13
C:\Users\YASWANTH\DAA>
```

4 TIME COMPLEXITY:

	$T(n) = T(n/2) + 1$
	$a = 1, b = 2, k = 0, p = 0$
	$f(n) = 1 \rightarrow O(1)$
	By calculation
	$n \log_2 b$
	$n \log_2 2$
	$n^{\log_2 b}(n) = n^{\log_2 2}(1) = 1$
	So by Master Theorem case 2:
	$T(n) = O(\log n)$
	Heapsort = $O(\log n)$
	$T(n) = O(n) + O(\log n)$
	$T(n) = O(n \log n)$
	For max-heap sort
	Best = Avg = worst = $O(n \log n)$
	worst(n)
	$T(n/2) + O(1)$
	$a = 1, b = 2, f(n) = 1$
	$n \log_2 2$
	$T_{\text{worst}}(n) = O(n \log n)$

5 Conclusion:

The Max-Heap Sort algorithm was successfully implemented and executed. The given elements were sorted in ascending order, and the time complexity was analyzed for the best, average, and worst cases. Thus, the implementation of the Max-Heap Sort algorithm was successfully done.